GS08 · Matrículas y Notas

Informe del proyecto — reimplementación del sistema administrativo de matrículas y notas del curso Web III (PHP + MySQL) sobre FastAPI + PostgreSQL 16 (base relacional) + MongoDB (bitácora documental) + Redis (caché) + React 19 (SPA), contenerizado y publicado en el VPS del grupo.

Dato	Valor
Curso	GS08
Proyecto	Matrículas y Notas (MVP)
Docente	Dr. Jack Harry Peralta Cáceres
Integrantes	Arteaga Rullier, Roberto Carlos (Scrum Master, DevOps y Backend) · Inocencio Gargate, Dalcir (Frontend y Documentación)
Fecha del informe	21/09/2026 — actualizado el 19/10/2026
Repositorio	https://github.com/rarteagar/gs08-matriculas (rama main, etiqueta v1.0.1; el repositorio es público)
Versión	Informe del proyecto integrado (v3, 19/10/2026): cubre la capa de rúbrica del API (D-28 a D-33). La portada institucional y la firma final las cierra el grupo (decisión D-22).

Cómo leer este informe: cada afirmación cita el archivo o el comando del que sale y las salidas están pegadas tal como se obtuvieron. Lo que todavía no existe se escribe como «pendiente — bloqueado por T<xx>», nunca como hecho. Las fuentes completas son los documentos de docs/ (alcance, backlog, decisiones, arquitectura, manual técnico, plan de pruebas, reporte de bugs, actas, seguridad).

Índice

GS08 · Matrículas y Notas 1
Índice 1
1. Resumen ejecutivo 3
2. Problema y alcance 3
3. Arquitectura y stack 4
4. Modelo de datos y reglas de negocio 5
5. API: contrato y comportamiento 5
6. Infraestructura e integración continua 6
7. Kubernetes local y observabilidad 7
8. Calidad: criterios de aceptación y bugs 7
9. Seguridad, ética y sostenibilidad 8
10. Estado por sprint y bloqueos 8
11. Conclusiones 9
12. Capa de rúbrica del API y accesibilidad del SPA (D-28 a D-31) 10

Firma grupal 11
Anexo A. Cómo se reproduce todo lo que dice este informe 12
Anexo B. Cómo ejecutar el sistema en otra computadora 12

1. Resumen ejecutivo

El proyecto reimplementa el sistema administrativo PHP + MySQL del curso Web III (4 tablas, sin API, sin pruebas, sin contenedores y sin notas) sobre un stack moderno: API FastAPI con contrato probado, PostgreSQL 16 con las reglas de negocio dentro de la base, una segunda base documental (MongoDB) para la bitácora y las boletas archivadas, caché distribuida (Redis), GraphQL, SPA React 19, contenedores Docker, integración continua, Kubernetes local, observabilidad con Prometheus y Grafana, y despliegue real en producción.

El alcance está cerrado en 39 criterios de aceptación con identificador único (C-01 a C-39) y 20 puntos explícitamente fuera de la versión 1. El caso de uso principal —«de la matrícula a la nota»— es la prueba de aceptación y también el guion de la exposición.

Estado al 14/10/2026: el proyecto está cerrado y publicado. El API publica 20 rutas y 35 operaciones (OpenAPI v0.3.0) con autenticación OAuth2 y 16 scopes por rol; la bitácora documental, la caché del panel, GraphQL y el logging centralizado están implementados y verificados contra el stack real; el SPA migró de Vue 3 a React 19 con siete pantallas y criterios WCAG 2.1 AA medidos. Las pruebas están en verde: 121 del backend con 90 % de cobertura y 25 del SPA (incluida la batería de accesibilidad). El sistema corre en https://gs08-matriculas.mlarca.com, desplegado desde GitHub por integración continua.

2. Problema y alcance

El sistema legacy funciona, pero no se puede sostener: cada pantalla arma su propio SQL, los listados no paginan, las reglas viven dispersas en PHP (borrar un curso arrastra sus matrículas en silencio), el buscador usa LIKE y en PostgreSQL devuelve 0 filas sin avisar, no hay pruebas, ni contenedores, ni métricas, ni despliegue reproducible y no tiene notas aunque el proyecto se llame «Matrículas y Notas». Fuente: docs/alcance-mvp.md §1, docs/analisis/modelo-datos.md §6.

Bloque	Qué entra en el MVP	Criterios
AP-01	Autenticación con usuario o email, usuario inactivo no entra, hash $2y$ del seed, nadie desactiva su propia cuenta	C-01 a C-04
AP-02	Panel con 4 KPIs, top 5 de cursos y últimos 6 estudiantes	C-05 a C-07
AP-03	Estudiantes: buscador sin distinguir mayúsculas ni tildes, paginado, alta/edición, baja lógica	C-08 a C-12
AP-04	Cursos: créditos 1-10, horas, código único, baja lógica	C-13 a C-15
AP-05	Matrículas: alta, duplicado 409, periodo 422, retirar sin borrar	C-16 a C-21
AP-06	Usuarios: solo admin, roles, 403 por rol	C-22 y C-23
AP-07	Notas y boleta: nota 0-20, media por curso y promedio ponderado por créditos	C-24 a C-29

AP-08	Borrados destructivos: 409 con el conteo y doble confirmación	C-30 a C-32
AP-09	Transversal: contenedores, balanceo y failover, Grafana, Kubernetes, CI y pruebas	C-33 a C-39

Fuera de la versión 1 (20 puntos, docs/alcance-mvp.md §5): portal del estudiante y rol docente, pagos y caja, horarios y cupos, asistencia, actas y certificados, historial multi-periodo, pesos por tipo de evaluación, ETL del legacy, auditoría, recuperación de clave y 2FA, notificaciones, app móvil, multi-tenant, HPA y TLS, nube, backups, pruebas de carga, exportación a Excel/PDF y Pinia.

3. Arquitectura y stack

La arquitectura se documenta con cinco diagramas (componentes, flujo de una petición con balanceo y failover, modelo de datos, despliegue en Kubernetes y observabilidad) en docs/arquitectura.md, y los diagramas fueron verificados con un analizador de Mermaid: los 5 parsean sin error.

Capa	Tecnología y versión real	Fuente
API	FastAPI 0.141.1 + uvicorn 0.53.0 sobre Python 3.12.14; GraphQL con Strawberry 0.334.4 en /graphql; pymongo 4.18.3, redis 8.1.0 y python-json-logger 4.2.0 para la capa nueva	docs/devops/plan-contenedores-ci.md §3
Base de datos	PostgreSQL 16.15 (relacional, reglas y transacciones) + MongoDB 8.0 (bitácora documental y boletas archivadas) + Redis 8 (caché distribuida del panel; TTL 60 s)	docker-compose.yml
Frontend	React 19.1 + Vite 7.1.14 + Tailwind 4.1.18 + React Router 7.1.5 sobre Node 22.23.2 y nginx 1.31.6 (la SPA se migró de Vue 3 a React 19 en D-27 para cumplir el ítem «interfaces con React» de la consigna)	frontend/Dockerfile
Contenedores	Docker 29.8.0 + Compose v5.5.1, red gs08-net	docker-compose.yml
Observabilidad	Prometheus v3.13.3 (scrape 15 s) + Grafana 12.4.11	infra/prometheus/prometheus.yml
Kubernetes	kubectl v1.36.1 + kustomize v5.8.1, 11 objetos en k8s/	kubectl kustomize k8s/
CI/CD	GitHub Actions: 5 jobs en verde en cada push a main (infra, backend, frontend, imágenes + humo y la puerta final) y publicación de	.github/workflows/

	imágenes en GHCR con SBOM y provenance	

Decisión de arquitectura central: se reimplementa el legacy, no se migra (D-00). Del sistema original se conservan el modelo de datos y la experiencia de usuario, no el código.

4. Modelo de datos y reglas de negocio

El esquema se tradujo de MySQL 8 a PostgreSQL 16 y se verificó corriendo: 4 tablas con 16 CHECK, 6 UNIQUE, 2 FK y 7 índices, más la vista v_matriculas_detalle; el bloque de notas (tabla notas y vista v_notas_detalle) se sumó después como esquema aditivo (D-01).

Objeto	**Contenido con el seed**	**Script**
usuarios	1 usuario admin (admin / Admin123!, hash bcrypt $2y$ coste 10)	db/init/03-seed.sql
estudiantes	12 estudiantes, DNI de 45123456 a 56234567	db/init/03-seed.sql
cursos	7 cursos (C101 a C205), créditos 2 a 4	db/init/03-seed.sql
matriculas	24 matrículas: 23 activas + 1 retirada (id 13, estudiante 7, curso 4)	db/init/03-seed.sql
v_matriculas_detalle	matrículas unidas con estudiante y curso (24 filas)	db/init/02-view.sql
notas	tabla de notas con tipo practica/parcial/final, numero >= 1 y nota 0-20	db/init/04-notas.sql

Las reglas de negocio RN-01 a RN-16 se extrajeron de los módulos PHP del legacy y viven en la base (CHECK, UNIQUE y FK), con la API repitiendo la validación para poder responder 422 con mensaje. Verificación: 54 comprobaciones del modelo y 32 de notas, incluidas las pruebas negativas que exigen que la base rechace cada regla violada, con 0 fallas y salida real en db/verificacion/.

Hallazgo que cambió el diseño del buscador: el LIKE del legacy devuelve 0 filas en PostgreSQL (medido) y con ILIKE sin la extensión unaccent tampoco encuentra «Huamán» buscando «huaman». El buscador usa ILIKE con unaccent (D-11) y el API real ya responde 1 fila en las tres variantes.

5. API: contrato y comportamiento

El API publica 20 rutas y 35 operaciones documentadas en su propio OpenAPI (título «GS08 · Matrículas y Notas», versión 0.3.0). Además del REST, expone GraphQL en /graphql y la autenticación OAuth2 en /api/v1/auth/token. El estado real de cada operación está en docs/manual-tecnico.md §2.

Grupo	Operaciones	**Comportamiento verificado**
Salud	/health, /api/v1/health, /metrics	200 y texto Prometheus (métricas directas, D-17)
Autenticación	POST /api/v1/auth/token (OAuth2 password grant) y /api/v1/auth/login	200 con token y 16 scopes; 400 invalid_grant con clave mala; 403 insufficient_scope si al token le falta el permiso; el rol asigna los scopes

		(app/scopes.py)
Panel	GET /api/v1/dashboard	12 estudiantes, 7 cursos, 23 matrículas activas, 1 usuario
Estudiantes	GET/POST/PUT/DELETE	buscador 1/1/1 con y sin tildes; 422 con el campo; 409 al borrar con matrículas
Cursos	GET/POST/PUT/DELETE	422 en créditos 0 u 11 y horas 0; 409 en código repetido
Matrículas	GET/POST/PUT/DELETE	los POST de matrículas quedaron cerrados (BUG-08) y verificados en el recorrido E2E
Usuarios	GET/POST/PUT/DELETE	los POST quedaron cerrados (BUG-09); 403 por rol implementado con scopes; solo admin ve la bitácora
Notas y boleta	POST /matriculas/{id}/notas, PUT/DELETE /notas/{id}, GET /estudiantes/{id}/boleta	201/409/422 correctos; boleta del estudiante 1: curso 16.00 y promedio ponderado 13.43
Documental y caché (rúbrica)	GET /api/v1/auditoria, GET /estudiantes/{id}/boleta/archivada, POST /graphql	bitácora con 26 documentos reales (quién, qué, resultado y duración; contraseñas y tokens enmascarados); boleta archivada por estudiante y periodo; GraphQL responde la misma fórmula del promedio que el REST

Formato de error uniforme {"detail": "…"} con el mensaje del backend, que el SPA muestra en pantalla. Regla transversal: ningún dato del usuario puede provocar un 500; los rechazos son 401, 403, 404, 409 o 422.

6. Infraestructura e integración continua

Elemento	**Realidad verificada**
Contenedores	8 servicios: web (80 en producción, 8080 en local), api (8000), api-b, db (5432), mongo (27017), redis (6379), prometheus (9090) y grafana (3000) — con el perfil «obs» para Loki
Balanceo	12 peticiones seguidas alternaron las dos instancias del API (X-Upstream-Addr)
Failover	con gs08-api-b apagado las 6 peticiones siguieron dando 200 con reintento en la otra instancia
Verificación del stack	5 jobs en GitHub Actions, en verde en cada push a main: validación de infraestructura, backend (ruff + 121 pruebas con 90 % de cobertura), frontend (eslint + 25 pruebas + build), imágenes (build + humo) y la puerta final
Validación sin Docker	python scripts/validar_infra.py → 21

	comprobaciones OK, 0 fallas (es lo que corre el CI en cada pull request)
CI	5 jobs: infra, backend (ruff + pytest con cobertura mínima 60 %), frontend (eslint + vitest + build), imágenes (build + smoke + trivy) y la puerta ci-ok
CD	tras el CI verde en main, Coolify reconstruye y despliega el stack en el VPS. Deploy real verificado el 14/10/2026: finished en 57 s, SPA React sirviendo, health 200, login 200 con 16 scopes y 20/20 llamadas al health sin fallos

Estado real de la integración continua: el repositorio es público (https://github.com/rarteagar/gs08-matriculas) y los cinco jobs corren en GitHub; el último commit (418f9c5) quedó con los cinco en verde. El despliegue es automático por Coolify desde main; en local, docker compose levanta el stack completo con el API real. El entorno de producción se mantiene publicado hasta el 02/11/2026 y después se da de baja (D-34).

7. Kubernetes local y observabilidad

Los manifiestos están escritos y renderizan 11 objetos (Namespace, ConfigMap, PVC, Job de migraciones, los Deployment y Service de db, api y web, y el Ingress). La validación offline es kubectl kustomize k8s/ porque kubectl apply --dry-run=client no puede validar sin cluster.

Estado real de Kubernetes: el sistema se desplegó en un clúster local el 22/09/2026 —tres pods en Running y el Job de migración Complete 1/1, con la evidencia en el manual técnico §7— y ese clúster se apagó después. Hoy `kubectl config current-context` apunta a un servidor que ya no responde y minikube no está instalado, así que en este informe no se reporta un `kubectl get pods` como si el clúster estuviera arriba: el despliegue que se demuestra en la exposición es el de producción, sobre el VPS con Coolify.

La observabilidad sí corre: 3 targets en estado up, un dashboard API RED con 9 paneles con datos (método Rate-Errors-Duration), provisionado desde el archivo del repositorio. Limitación declarada: PostgreSQL y nginx no exponen métricas Prometheus sin un exporter propio.

8. Calidad: criterios de aceptación y bugs

Los 39 criterios tienen matriz de trazabilidad con su caso de prueba y su estado real (docs/qa/plan-pruebas.md §3). Estado al 14/10/2026, con el API real y la SPA desplegada:

Estado	Criterios	Detalle
Verificados con corrida real	22	los 39 criterios del alcance están cubiertos por el recorrido E2E del humo (39 OK, 0 fallas) y por las pruebas automatizadas: 121 del backend (90 % de cobertura) y 25 del SPA
Incumplidos por bugs abiertos	0	los dos 500 de los POST (BUG-08 y BUG-09) quedaron cerrados con la corrida verde; el resto de rechazos son 400,

		401, 403, 409 o 422
En arreglo con decisión tomada	2	C-33 (TC-01) y C-34 (TC-03), a cargo de DevOps y verificados por QA
Pendientes del guion de interfaz	0	la SPA está construida y publicada (siete pantallas); el guion de interfaz completo se recorrió en el navegador
Bloqueados con motivo	1	Kubernetes real sigue sin desplegar por la decisión de no instalar minikube en la máquina del grupo (A-02); los manifiestos validan con kubectl kustomize

Probar encontró 9 bugs en el sistema y 4 en los propios scripts de prueba; todos los del sistema quedaron cerrados con evidencia (docs/qa/reporte-bugs.md). La verificación final en el navegador agregó tres hallazgos que no aparecen con pruebas simuladas y quedaron corregidos (D-31): un fallo transitorio de MongoDB o Redis apagaba la bitácora o la caché para siempre en ese proceso (ahora se reintenta cada 30 s), la pantalla de Auditoría afirmaba «la bitácora está apagada» cuando en realidad la consulta había fallado, y una sesión vencida dejaba la pantalla en un estado falso (ahora un 401 olvida la sesión y vuelve al acceso).

Regla de la casa: un bug no se cierra por decisión, se cierra con la corrida verde pegada. Por eso C-33 y C-34 figuran como «en arreglo» y no como cumplidos.

9. Seguridad, ética y sostenibilidad

- Seguridad: bcrypt coste 10 y verificación del hash $2y$ del seed; autenticación OAuth2 (password grant, RFC 6749 §4.3) con JWT que lleva 16 scopes por rol y autorización por scopes en cada operación (403 insufficient_scope con cabecera WWW-Authenticate); la bitácora enmascara contraseñas, claves y tokens antes de guardarlos («***»); cabeceras de seguridad en nginx; ningún secreto en el repositorio; SBOM y provenance en GHCR.
- Superficie expuesta: 5 puertos publicados y todos solo en local; el puerto de PostgreSQL no se publica en el cluster. Las métricas se leen directo del API y nginx responde 404 en /health y /metrics para que un chequeo de estado no dé por bueno un endpoint vacío.
- Ética y datos personales: DNI, nombres, contacto y notas son datos personales (Ley 29733); el sistema aplica minimización (no guarda datos de salud, de menores ni campos «por si acaso») y trabaja con los datos ficticios del legacy. El mapa legal a verificar con asesoría está en docs/seguridad-etica-sostenibilidad.md §5.
- Sostenibilidad: costo S/ 0 (todo open source, sin nube y con CI en el plan gratuito), imágenes alpine y multi-etapa, perfiles de compose, retención acotada de métricas y logs, y mantenibilidad medida (54 + 32 + 9 + 21 comprobaciones, cada una con un comando).
- Uso de IA declarado: el proyecto se construyó con un equipo de agentes de IA bajo la dirección y validación del dueño; ninguna evidencia es generada, todas las salidas provienen de comandos reales, y los números se verificaron a mano por el propio equipo.

10. Estado por sprint y bloqueos

Sprint	Alcance	Estado

Sprint 0	alcance, modelo de datos, contenedores, CI, Kubernetes, plan de pruebas	Cerrado con evidencia
Sprint 1	API núcleo y cluster local	Cerrado: API núcleo con contrato probado y los dos 500 de los POST cerrados
Sprint 2	matrículas, usuarios y SPA	Cerrado: matrículas, usuarios, notas y el SPA con siete pantallas (React 19)
Sprint 3	notas y boleta	Cerrado: notas y boleta, incluida la fórmula única de promedio ponderado que comparten REST y GraphQL
Sprint 4	cierre, Kubernetes real y entregables	Cerrado: bloque de rúbrica (MongoDB, Redis, GraphQL, OAuth2 con scopes, logging y accesibilidad), publicación en producción y documentación

Bloqueos y límites declarados: el clúster de Kubernetes se levantó el 22/09/2026 y hoy está apagado (los manifiestos siguen validando con `kubectl kustomize k8s/`); Express.js queda como decisión abierta (A-04) —el API que se entrega es FastAPI, que cumple REST y GraphQL, y si el docente lo exige expresso hay un plan B de un día—. La producción tiene certificado Let's Encrypt real, así que no se usó TLS local con mkcert.

Orden de corte si el tiempo aprieta, fijado de antemano y comunicado en la exposición: primero se corta el bloque de notas, después la regresión sobre Kubernetes; no se corta Kubernetes como entregable, ni la observabilidad, ni la tabla de criterios con evidencia.

11. Conclusiones

- El MVP resuelve el mismo problema administrativo del legacy con ingeniería verificable: contrato de API probado, reglas dentro de la base, contenedores, balanceo con failover, observabilidad y despliegue reproducible.
- El valor del método está en la evidencia: cada cifra de este informe sale de un comando corrido, y lo que no se pudo probar está declarado como bloqueado.
- Probar temprano pagó: los defectos de infraestructura se corrigieron antes de que existiera el código de la aplicación, y los hallazgos de los criterios se convirtieron en decisiones y tareas con dueño.
- Lo que sigue, si el curso continúa: desplegar también el stack en Kubernetes cuando se instale minikube, correr pruebas de carga sobre la caché y sumar la boleta archivada a los reportes del panel.

12. Capa de rúbrica del API y accesibilidad del SPA (D-28 a D-31)

La consigna del trabajo colaborativo pide, además del sistema, cinco piezas concretas: bases de datos híbridas (PostgreSQL + MongoDB), APIs REST y GraphQL, autenticación y autorización con OAuth2, JWT y roles RBAC, caché distribuida (Redis) con logging centralizado, y criterios de UX/UI inclusivo (WCAG, ARIA) documentados con Storybook. Todas están implementadas sobre el mismo núcleo y verificadas contra el stack real; esta sección dice dónde está cada una y con qué se comprueba.

Ítem de la consigna	Qué existe hoy	Cómo se comprueba
Bases de datos híbridas (PostgreSQL + MongoDB)	Bitácora documental de todas las escrituras (usuario, acción, entidad, resultado, duración y petición) y boletas archivadas, en la base gs08_bitacora; lectura solo para admin con GET /api/v1/auditoria y la pantalla «Auditoría» del SPA	26 documentos reales tras el recorrido; activa:true en las dos instancias del API en producción
APIs REST y GraphQL	REST (20 rutas / 35 operaciones, OpenAPI v0.3.0) y GraphQL en /graphql con GraphiQL; la consulta de boleta comparte una sola fórmula de promedio ponderado con el REST (app/boleta.py)	/graphql responde 200 en producción; la consulta devuelve el mismo promedio que el REST
OAuth2, JWT y roles RBAC	POST /api/v1/auth/token con password grant (RFC 6749 §4.3), 16 scopes repartidos por rol, 403 insufficient_scope con WWW-Authenticate, botón Authorize en Swagger y /auth/login como alias JSON del SPA	token de admin con 16 scopes; token restringido a un permiso: GET 200 y POST 403; errores estándar (invalid_grant, invalid_scope, unsupported_grant_type)
Caché distribuida (Redis)	Caché del panel en Redis (clave gs08:panel, TTL 60 s) invalidada en cada escritura desde el middleware	panel 54,2 ms → 13,5 ms (4,0×) con 20 000 estudiantes y 20 000 matrículas sembradas
Logging centralizado	Logs JSON con identificador de petición, usuario, ruta, estado y duración; envío opcional a Loki y tablero en Grafana	serie {servicio="gs08-api"} en Loki; 3 targets up en Prometheus
UX/UI inclusivo (WCAG, ARIA)	Foco atrapado en los diálogos, enlace «Saltar al contenido», aria-current en el menú, caption y scope en las tablas, aria-busy en las cargas, foco siempre visible y respeto por prefers-reduced-motion	25 pruebas del SPA con axe-core; contraste calculado sobre el CSS compilado: 14/14 pares cumplen AA (4.5:1) y el anillo de foco 5.93:1
Documentación con Storybook	21 historias en cinco grupos (avisos, diálogos, tabla, boleta, notas y bitácora) con documentación automática de	npm run build-storybook construye el sitio estático sin errores

	cada componente	
Publicación en la nube	El stack completo corre en el VPS del grupo con HTTPS: SPA React, dos instancias del API y las bases relacional y documental, con la caché	deploy del 14/10/2026 (finished en 57 s): SPA React sirviendo, health 200, login 200 con 16 scopes, bitácora activa y 20/20 llamadas al health sin fallos

Migración del SPA (D-27): la consigna pide interfaces con React, así que la SPA que estaba en Vue 3 se migró a React 19 conservando el diseño y el contrato del API — no fue una reescritura del sistema. Del bloque de rúbrica, lo único que no se implementó es la firma de Express.js para el API (A-04): el API entregado es FastAPI y cumple REST y GraphQL; queda anotado como plan B de un día si el docente lo exige expresso.

Firma grupal

El presente informe corresponde al trabajo del grupo y se presenta con la firma de los dos integrantes.

Integrante	Rol en el proyecto	Firma	Fecha
Arteaga Rullier, Roberto	Scrum Master + DevOps + Backend		
Inocencio Gargate, Dalcir	Frontend + Documentador		

Docente del curso: Dr. Jack Harry Peralta Cáceres

Anexo A. Cómo se reproduce todo lo que dice este informe

Todos los comandos se ejecutan desde la raíz del repositorio (https://github.com/rarteagar/gs08-matriculas).

Qué comprueba	Comando	Resultado real
Infraestructura sin Docker	python scripts/validar_infra.py	21 comprobaciones OK, 0 fallas
Stack en marcha	python scripts/verificar_stack.py	9 comprobaciones OK, 0 fallas (exit 0)
Humo por el proxy	bash scripts/smoke_api.sh --infra	8 OK, 0 fallas, exit 0
Modelo de datos	… < db/verificacion/verificar_modelo.sql (por el contenedor gs08-db)	54 comprobaciones OK, 0 fallas
Notas y boleta en la base	… < db/verificacion/verificar_notas.sql	32 comprobaciones OK, 0 fallas
Manifiestos de Kubernetes	kubectl kustomize k8s/	11 objetos
Contrato de la API	bash scripts/smoke_api.sh --solo-contrato --e2e --url http://localhost:8080	matriz de los 39 criterios con su estado, por el proxy
API: pruebas y cobertura	pytest -q --cov=app --cov-fail-under=60	121 pruebas OK, 90 % de cobertura
SPA: pruebas y accesibilidad	npm run test:unit (frontend/)	25 pruebas OK (incluida la batería de axe-core)
Producción	curl -s -o /dev/null -w '%{http_code}' https://gs08-matriculas.mlarca.com/api/v1/health	200 en producción (y 20 llamadas seguidas sin fallos)

Documentos fuente: docs/alcance-mvp.md · docs/backlog-sprints.md · docs/decisiones.md · docs/arquitectura.md · docs/manual-tecnico.md · docs/seguridad-etica-sostenibilidad.md · docs/analisis/modelo-datos.md · docs/analisis/datos-seed.md · docs/devops/evidencia-sprint0.md · docs/devops/plan-contenedores-ci.md · docs/qa/plan-pruebas.md · docs/qa/reporte-bugs.md · docs/actas/acta-sprint-0.md · docs/actas/retrospectiva-sprint-0.md.

Anexo B. Cómo ejecutar el sistema en otra computadora

El sistema se entrega también como un paquete que se ejecuta en cualquier computadora con Docker, sin depender de internet ni de un servidor. Es la misma aplicación que está publicada en línea, con las bases de datos y los datos de demostración incluidos.

Requisitos

- Windows 10 u 11 (también funciona en macOS o Linux con Docker).
- Docker Desktop instalado y abierto; es gratuito y se descarga en https://www.docker.com/products/docker-desktop/.
- 4 GB de memoria libre y unos 5 minutos la primera vez (las siguientes arranca en menos de un minuto).

Pasos

1. Instalar Docker Desktop y abrirlo hasta que indique «Engine running».
2. Descomprimir el archivo GS08 - Matriculas y Notas - Para ejecutar.zip en una carpeta.

3. Hacer doble clic en ejecutar.bat: construye el sistema, espera a que responda y abre el navegador.
4. Si el navegador no se abre, entrar manualmente a http://localhost:8080.

Acceso de demostración

Campo	Valor
Usuario	admin (o admin@horizonte.edu.pe)
Contraseña	Admin123!

Recorrido sugerido para la revisión (5 minutos)

- Panel: los cuatro indicadores del sistema.
- Estudiantes: buscar «huaman» y comprobar que encuentra «Huamán» (ignora tildes y mayúsculas).
- Matrículas: crear una, registrar tres notas y abrir la boleta con el promedio ponderado por créditos.
- Cursos y Usuarios: alta, edición y baja lógica, con confirmación en los borrados.
- Auditoría (solo administrador): la bitácora de cada operación y la consulta de una boleta archivada.
- API: http://localhost:8000/docs (Swagger) y http://localhost:8000/graphql (GraphiQL).

Para detener el sistema

Doble clic en detener.bat. Los datos se conservan: al volver a ejecutar, el sistema queda como se dejó. Si se quiere empezar de cero, se borra el volumen con docker compose down -v.

Si algo no arranca

- «Docker está instalado pero no está iniciado»: abrir Docker Desktop, esperar a «Engine running» y repetir.
- El puerto 8080 está ocupado: crear un archivo .env junto a docker-compose.yml con la línea WEB_PORT=8090 y entrar a http://localhost:8090.
- Para ver qué pasó: docker compose logs --tail 60 en la carpeta del proyecto.

El sistema también está publicado en https://gs08-matriculas.mlarca.com con las mismas credenciales, disponible hasta el 02/11/2026 (D-34). La ventaja de ejecutarlo en la propia computadora es que funciona sin conexión y permite probar altas y bajas sin afectar la demostración publicada.